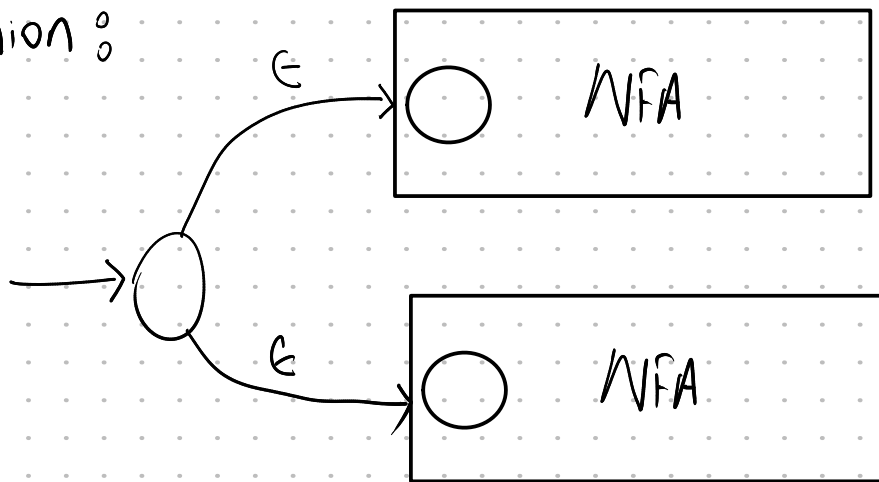


NFA For regular operation

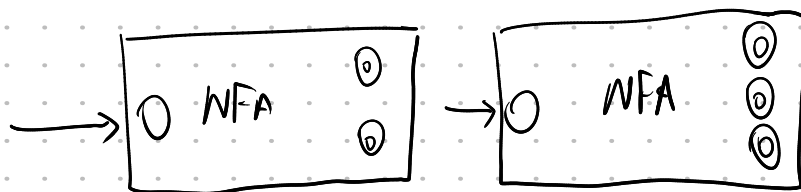
Union :



If it is in union, then one of the two machines will accept.
Which means by non determinism, it will make the right choice to either go up or down.

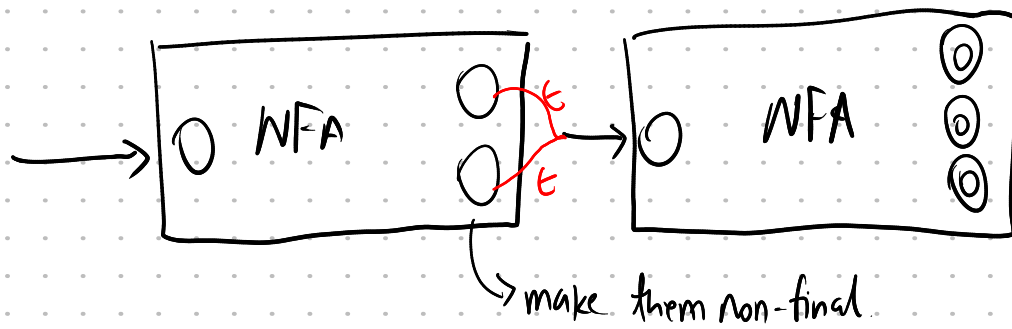
Concatenation.

In Union the order did not matter but for Concatenation it does.

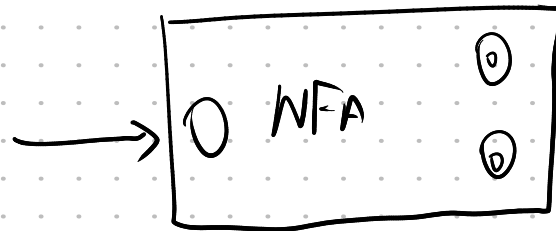


Idea was we can jump instantaneously from the first machine to the second one, but now we have that transition which are the epsilon transition. We only want to go from the ones that are the final state in the first, I want to read a string that the first one

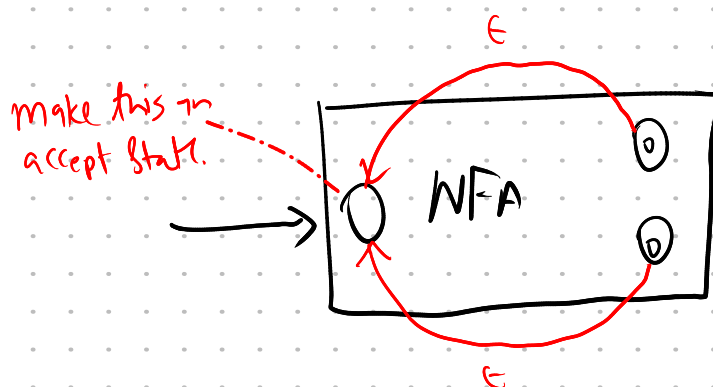
accepts and then jump to the second one where it reads rest of the string and maybe accepts.



Star:

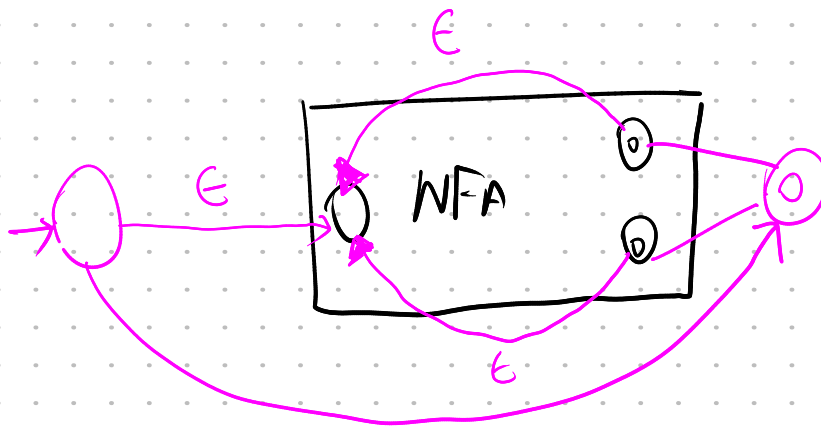
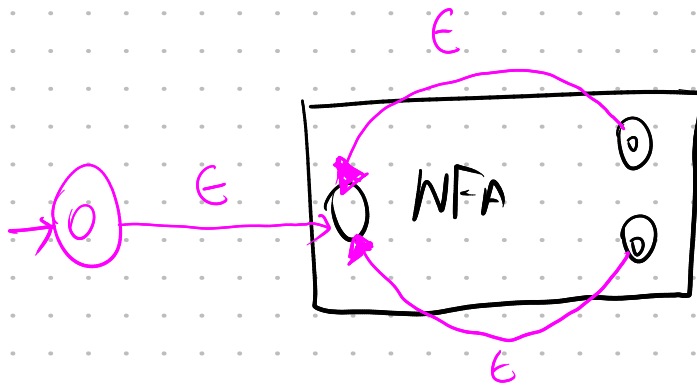


Once we go through the machine we pick a string out of the language, we want to be able to go back through again as many times as we want. The idea is to have an epsilon transition from the end to beginning. There's actually a problem with this. mainly because in the star of any language the empty string is contained in there.



Making the start state accept state will change the original machine's language.

So the idea is to not do this, make a new start state that is an accept state itself, this makes sure the behaviour of the original NFA is not changed, then from the accepting state go to the brand new starting state.



In DFA we did Union, Concatenation and Star we have not shown.

Lemma: Every regex can be converted into an NFA.

Regex R

1. $R = \emptyset \rightarrow \bigcirc$

2. $R = \epsilon \rightarrow \bigcirc \rightarrow \bigcirc$

3. $R = a \rightarrow \bigcirc \xrightarrow{a} \bigcirc$

4. $R = R_1 \cup R_2$

5. $R = R_1 R_2$

6. $R = (R_1)^*$

} This is actually a proof that if you do the Closure and Complement thing for NFA you will fail because these 2 are not complements of each other, because the first has zero string in its language and the second has one string.

Conditions

The NFA derived from Thompson's Construction must follow some conditions:

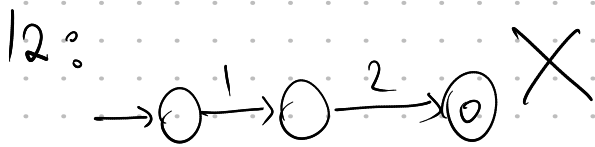
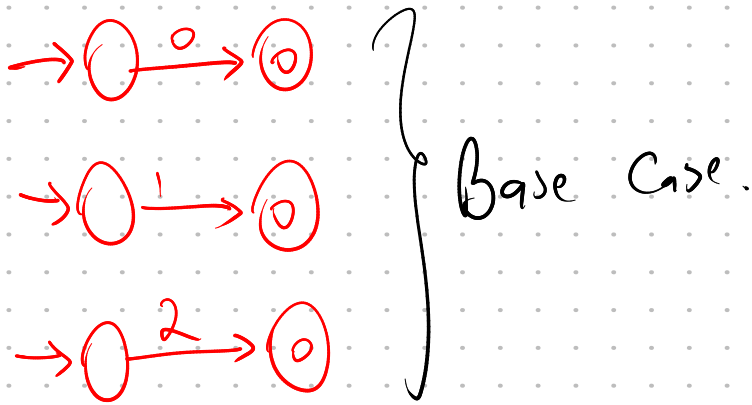
1. There can be only one start state.
2. The start state cannot be accessible from any states i.e. it cannot have transitions to it from other states.
3. There can be only one final state. The final state cannot have any transition from it to other states.
4. The number of transitions leaving any state is at most two.

Example:

① $(0 \cup 12)^*$

We are working on a recursive structure because there are some base cases 1, 2, 3 and some recursive or inductive cases.

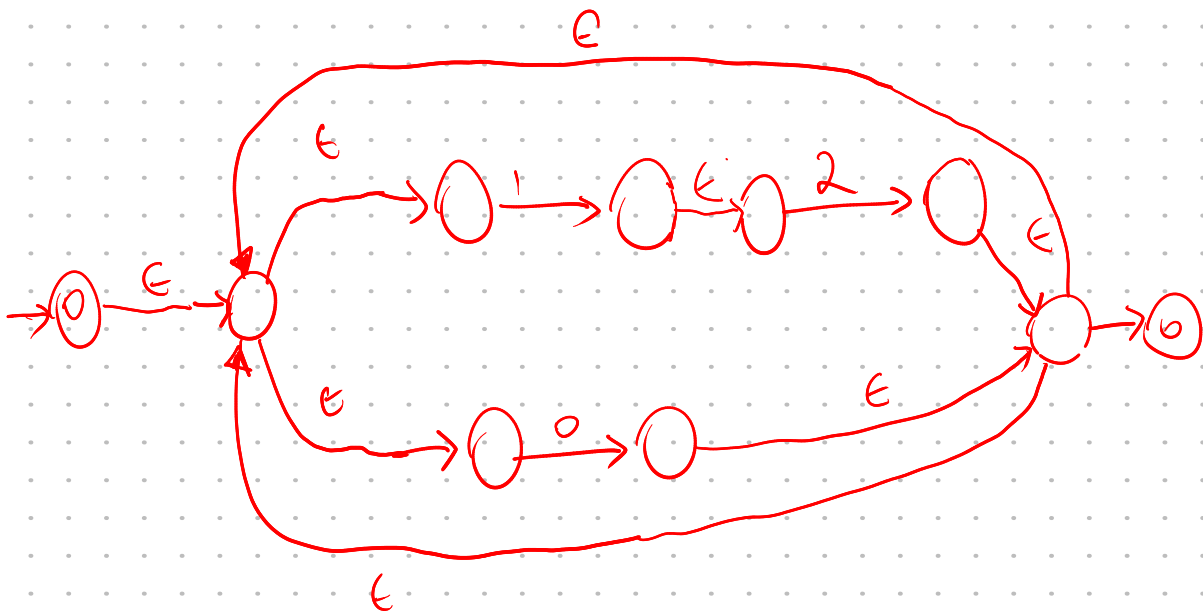
Focus on small regex and build bigger and bigger ones of those.



valid NFA, Not correct in terms of conversion.

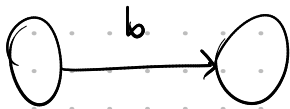
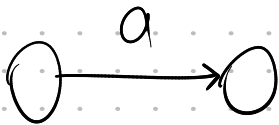


Epsilon transition from the first NFA final state to the second NFA start state.

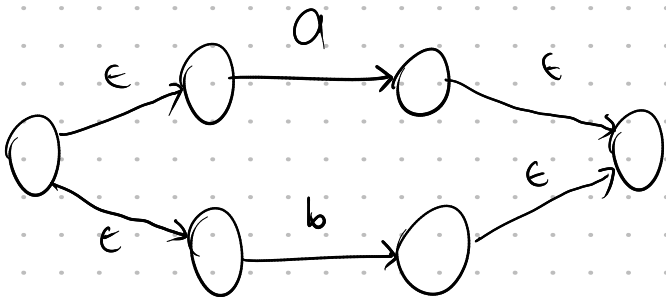


② $(a|b)^*abb$

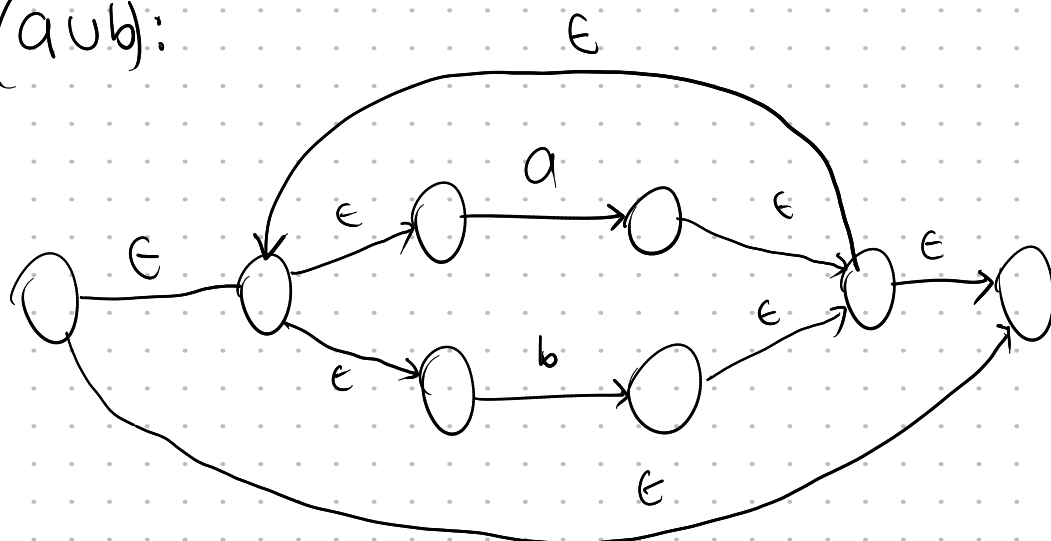
Base case :



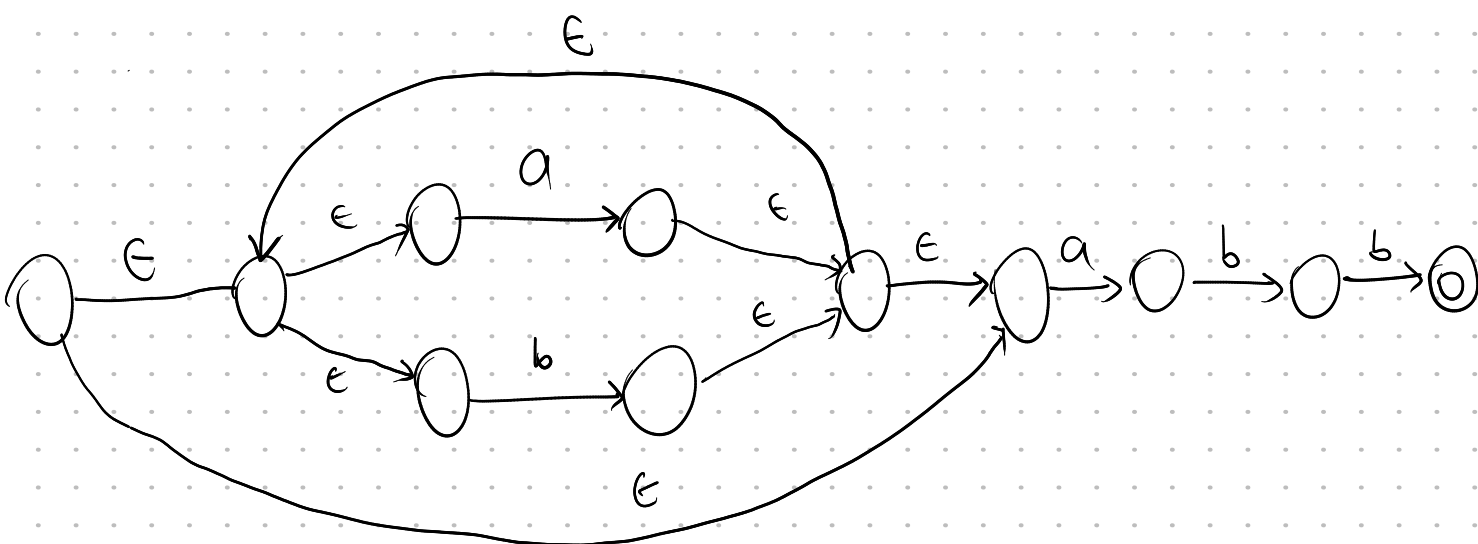
$a \cup b$:



$(a \cup b)^*$:

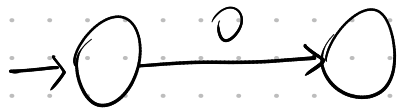


$(a/b)^* abb$

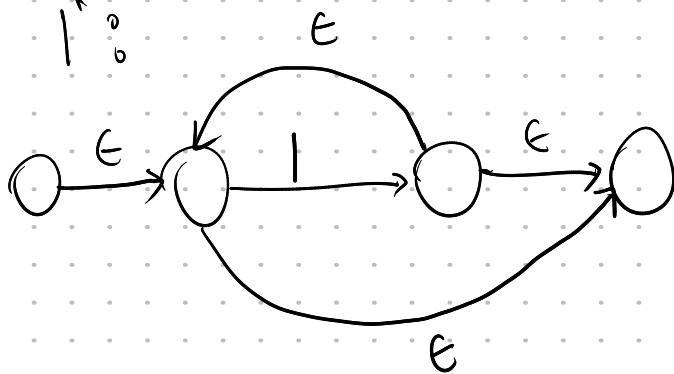


$$③ (0+11)^* + 0^*111^*$$

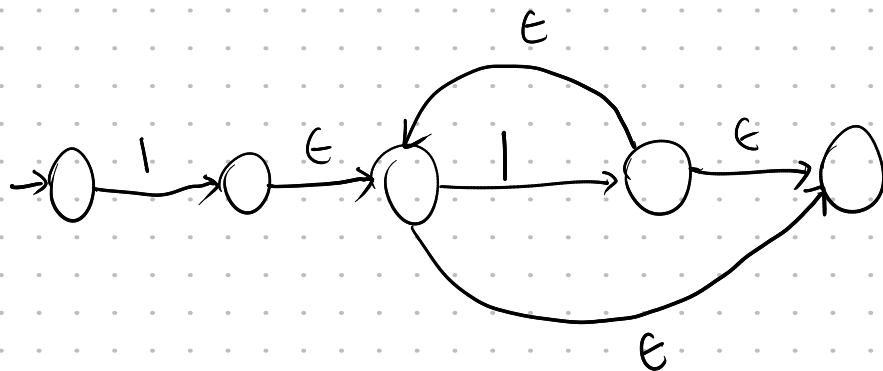
0^*



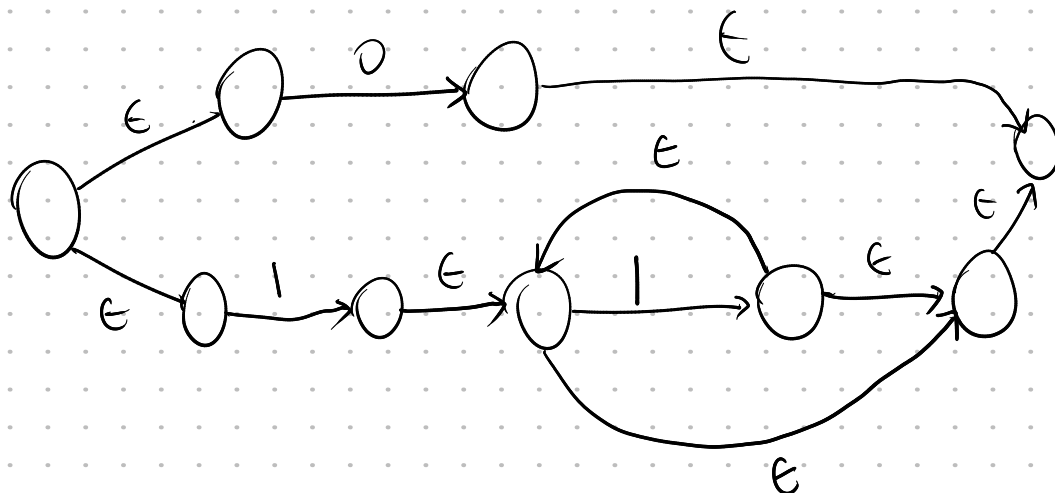
1^*



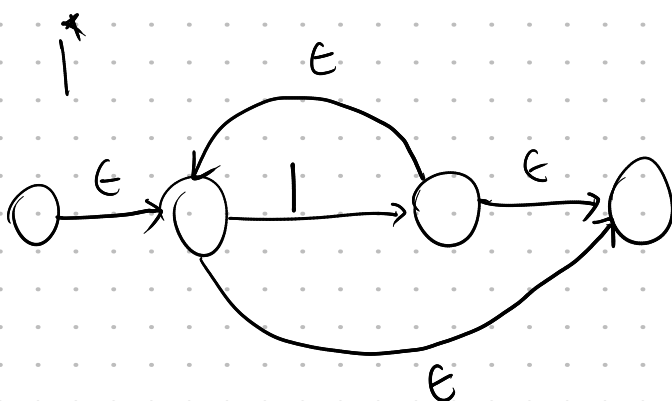
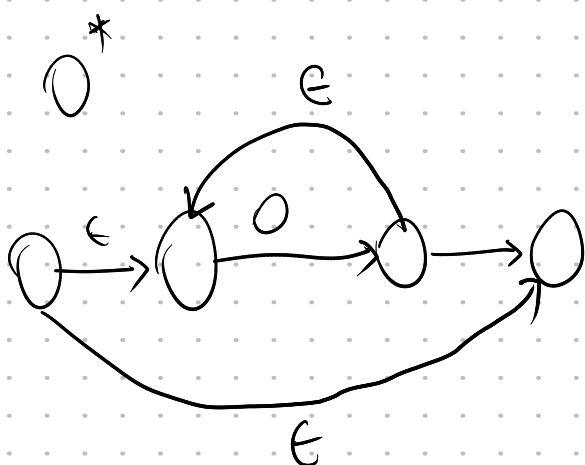
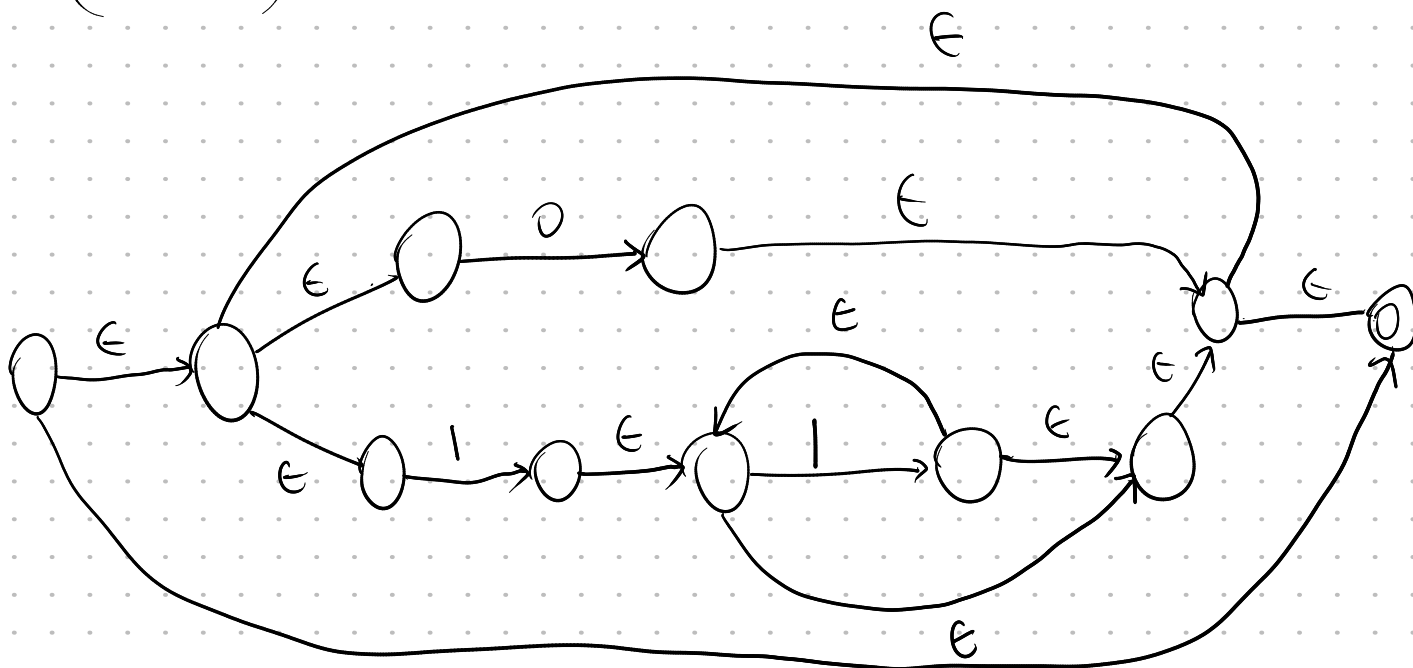
11^*



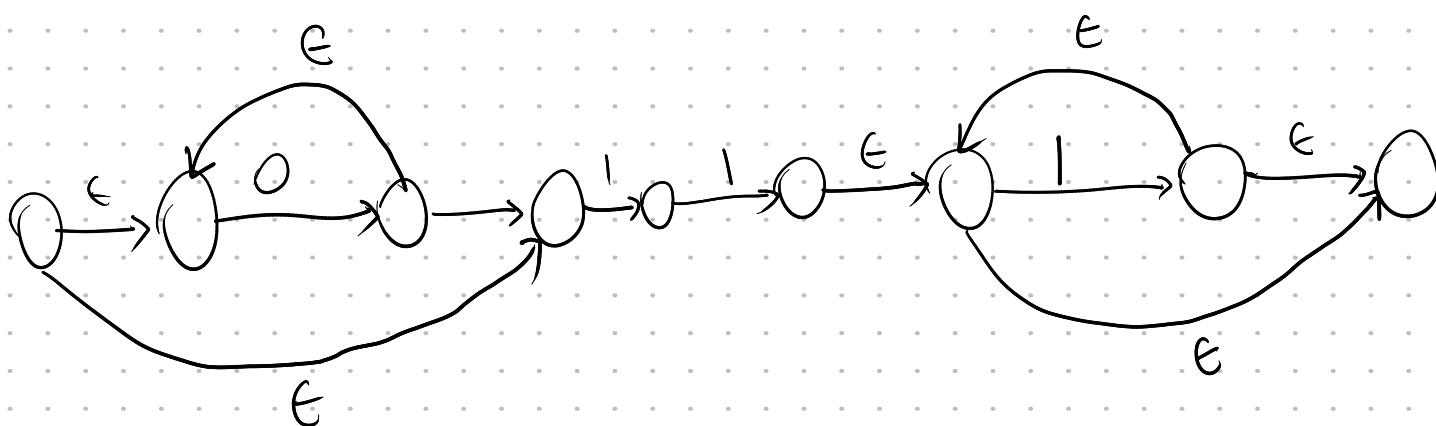
$(0+11^*)$



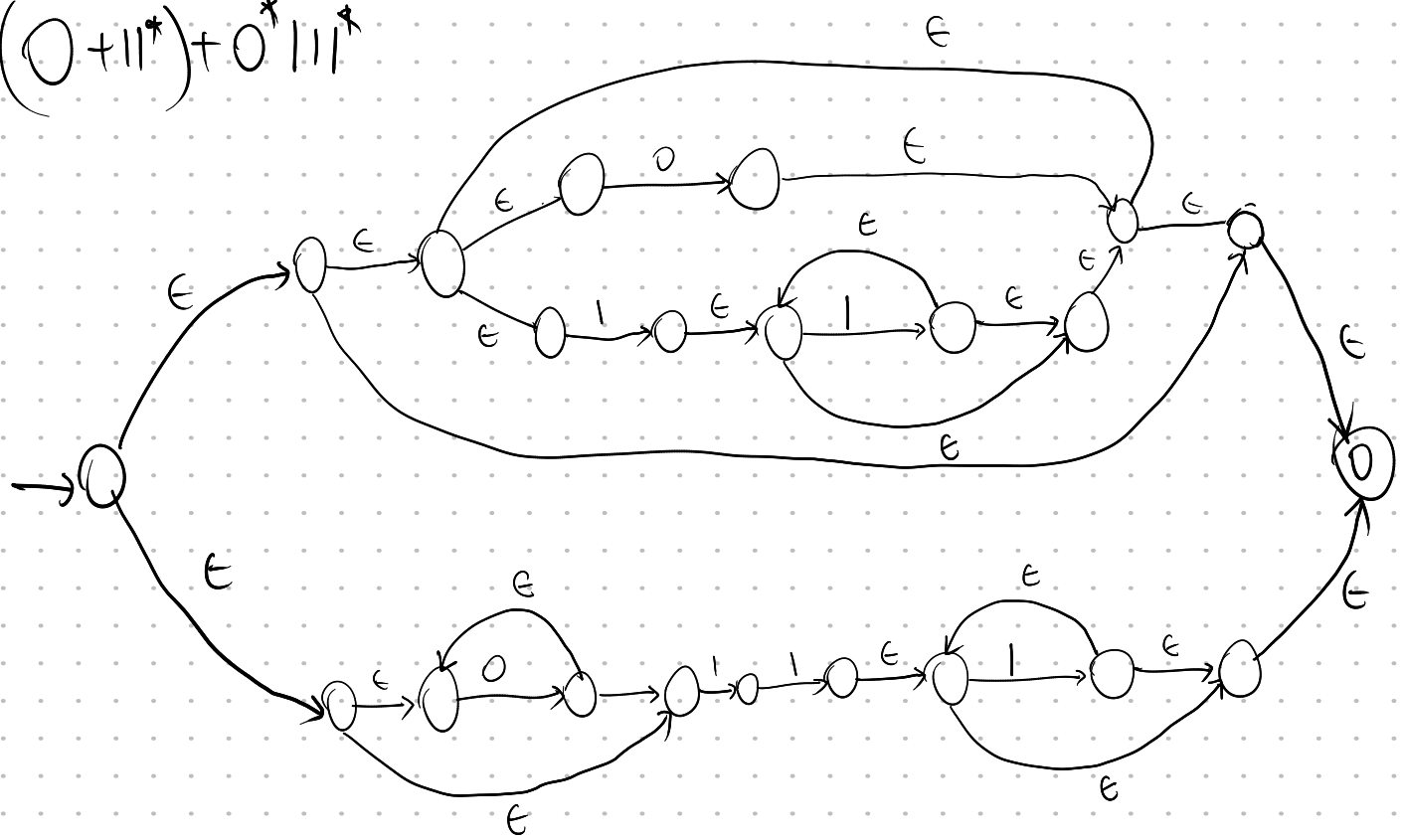
$(0+11^*)^*$



0^*111^*



$$(0+11^*)+0^*111^*$$



Solve:

$$(0+1)^*110(00+11)^*$$